PERKULIAHAN MINGGU 5 • 12S3101



Arsitektur Modern Web Framework & Alur Kerja MVC

Penerapan Pola Model-View-Controller pada Ekosistem Laravel 11.x

Mata Kuliah: Pemrograman dan Pengujian Aplikasi Web (12S3101)

Mata Kuliah: Pemrograman dan Pengujian Aplikasi Web (12S3101)

DOSEN PENGAMPU:

Chandro Pardede, S.Kom., M.Sc.

Institut Teknologi Del • Program Studi Sarjana Sistem Informasi

Learning Objectives & Capaian Perkuliahan

Kompetensi Utama Mahasiswa

- Memahami filosofi pemisahan kepentingan (*Separation of Concerns*).
- Menganalisis siklus hidup request (*Request Lifecycle*).
- Menavigasi struktur direktori bawaan Laravel 11.
- Mengimplementasikan alur CRUD secara terstandarisasi industri.
- Mengoperasikan tools modern (Artisan CLI & Vite).

Peta Capaian Praktis

- Paham alur request dari `public/index.php`.
- Mampu membuat Model, Migration, dan Controller.
- Mampu menghubungkan data ke Blade View.
- Mampu menjalankan build aset frontend produksi.
- Siap mengerjakan tugas praktikum laboratorium mandiri.

TARGET SUB-CPMK IT DEL

Mahasiswa mampu merancang dan menguji aplikasi web berbasis pola arsitektur modern secara modular, aman, dan efisien.

STANDAR MUTU REKAYASA

Lulusan Sistem Informasi IT Del diarahkan memiliki pemahaman mendalam tentang arsitektur internal framework, bukan sekadar menggunakan template.

Masalah Spaghetti Code pada PHP Tradisional

Karakteristik Kode Tak Terstruktur

- Percampuran query SQL di dalam tag HTML.
- Logika validasi formulir tersebar tanpa pola.
- Duplikasi kode koneksi basis data berulang.
- Ketiadaan modularitas mempersulit pengujian otomatis perangkat lunak.
- Biaya pemeliharaan (*maintenance cost*) membengkak drastis.

Contoh Anti-Pattern (Spaghetti PHP)

```php
<?php
// Anti-pattern: mahasiswa.php
$conn = mysqli_connect('localhost', 'root', '', 'db');
$res  = mysqli_query($conn, 'SELECT * FROM students');
while ($row = mysqli_fetch_assoc($res)) {
    echo '<tr><td>' . htmlspecialchars($row['nama']) .
'</td></tr>';
}
?>
```

DAMPAK TEKNIS (TECHNICAL DEBT)

Aplikasi berbasis spaghetti code rentan celah injeksi SQL, sulit dikembangkan tim besar, dan menjadi jebakan utang teknis.

MENGAPA INI BERBAHAYA?

Jika skema basis data berubah, pengembang harus menyunting ratusan baris file HTML satu per satu secara manual.

Mengapa Industri Menggunakan Web Framework?

Fondasi Rekayasa Perangkat Lunak

- Menyediakan arsitektur teruji dan terstandarisasi industri.
- Proteksi keamanan otomatis (CSRF, XSS, SQLi).
- Meningkatkan produktivitas tim melalui konvensi seragam.
- Pustaka komponen terintegrasi (Autentikasi, Mail, Queue).
- Mendukung otomatisasi pengujian unit dan fitur.

Prinsip Konvensi vs Konfigurasi

- *Convention over Configuration*: Aturan baku terdefinisi.
- Struktur folder konsisten di seluruh dunia.
- Mekanisme kolaborasi tim menjadi sangat mulus.
- Dokumentasi resmi menjadi acuan bersama developer.
- Mempercepat waktu rilis aplikasi ke pasar.

PARADIGMA FRAMEWORK MODERN

Framework membebaskan insinyur dari pekerjaan repetitif tingkat rendah sehingga fokus pada solusi logika bisnis klien.

STANDARISASI INDUSTRI

Framework bukan membatasi kreativitas kita, melainkan memastikan seluruh anggota tim menulis kode dengan standar kualitas yang sama.

Mengapa Memilih Laravel 11.x?

Keunggulan Ekosistem Laravel

- Sintaks elegan berorientasi kenyamanan pengembang.
- Eloquent ORM: Implementasi Active Record terbaik.
- Struktur kernel minimalis dan modular di versi 13(dilengkapi AI SDK).
- Komunitas global terbesar dan dokumentasi terlengkap.
- Solusi ekosistem siap pakai.

Inisialisasi Cepat Proyek

```
# Inisialisasi Proyek Baru Laravel 13 via Composer
composer create-project laravel/laravel portal-del

# Masuk ke direktori proyek
cd portal-del

# Menjalankan server pengembangan lokal
php artisan serve
```

POPULARITAS & ADOPSI GLOBAL

Menguasai lebih dari 65% pangsa pasar framework PHP modern dunia dengan 76.000+ bintang GitHub.

KESELARASAN DENGAN IT DEL

Penguasaan Laravel membekali mahasiswa dengan keahlian yang sangat diminati oleh industri perangkat lunak nasional dan multinasional.

Definisi & Filosofi MVC (Separation of Concerns)

Tiga Pilar Arsitektur

- *Model*: Pengelola data dan aturan bisnis.
- *View*: Penampil antarmuka pengguna grafis.
- *Controller*: Pengatur lalu lintas request dan response.
- Isolasi komponen mencegah efek samping perubahan kode.
- Kolaborasi paralel antara tim backend dan frontend.

Analogi Restoran Berbintang

- **Pelanggan (Browser)**: Membawa pesanan menu (Request).
- **Pelayan (Controller)**: Mencatat pesanan dan mengantar masakan.
- **Koki & Dapur (Model)**: Mengolah bahan makanan (Database).
- **Piring Saji (View)**: Menata hidangan estetis (Response).

PRINSIP SEPARATION OF CONCERNS

Model tidak tahu cara menggambar HTML; View tidak boleh query ke database; Controller menjadi jembatan netral di tengahnya.

PEMBAGIAN PERAN MUTLAK

Pelanggan tidak pernah masuk ke dapur koki, dan koki tidak melayani tamu secara langsung di meja makan.

Anatomi 'M' (Model): Tanggung Jawab Data

Pemrograman dan Pengujian Aplikasi Web (12S3101) • Chandro Pardede, S.Kom., M.Sc.

Tanggung Jawab Lapisan Model

- Merepresentasikan tabel basis data sebagai objek PHP.
- Menegakkan integritas tipe data (*type casting*).
- Mendefinisikan relasi antar entitas data (1-1, 1-N).
- Mengamankan mutasi massal via properti `$fillable`.
- Menyimpan logika query spesifik (*query scopes*).

Contoh Model Eloquent Sederhana

```php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    // Kolom yang diizinkan untuk mass-assignment
    protected $fillable = ['kode', 'nama', 'harga'];
    protected $casts    = ['harga' => 'integer'];
}
```

KONVENSI ORM ELOQUENT

Kelas model tunggal `Product` otomatis mewakili tabel jamak `products` di database MySQL/PostgreSQL.

KEKUATAN ACTIVE RECORD

Model adalah representasi data murni. Berkat Eloquent, kita dapat mengambil atau memfilter data tanpa menulis raw SQL.

Anatomi 'V' (View): Presentasi & Rendering

Pemrograman dan Pengujian Aplikasi Web (12S3101) • Chandro Pardede, S.Kom., M.Sc.

Tanggung Jawab Lapisan View

- Hanya berfokus pada visualisasi antarmuka pengguna.
- Menerima variabel siap saji dari Controller.
- Dilarang memuat query basis data langsung.
- Menggunakan mesin template Blade yang efisien.
- Fitur proteksi otomatis terhadap injeksi skrip XSS.

Contoh Template Blade Bersih

```
<!-- resources/views/products/index.blade.php -->
<h2>Katalog Produk IT Del</h2>

<ul>
    @foreach ($products as $item)
        <li>{{ $item->nama }} - Rp{{
number_format($item->harga) }}</li>
    @endforeach
</ul>
```

PRINSIP 'DUMB VIEW'

View harus tetap 'bodoh'. View hanya menampilkan data olahan Controller tanpa melakukan kalkulasi bisnis berat.

SINTAKS {{ }} OTOMATIS AMAN

Sintaks kurung kurawal ganda Blade otomatis memanggil htmlspecialchars() untuk menetralkan karakter berbahaya.

Anatomi 'C' (Controller): Otak Orkestrasi

Tanggung Jawab Lapisan Controller

- Menerima payload HTTP Request dari pengguna.
- Memicu validasi keabsahan data formulir masukan.
- Meminta pemrosesan data kepada lapisan Model.
- Memilih template View yang sesuai kebutuhan.
- Mengembalikan HTTP Response dengan status code tepat.

Contoh Controller Action Ringkas

```
namespace App\Http\Controllers;

use App\Models\Product;
use Illuminate\View\View;

class ProductController extends Controller
{
    public function index(): View {
        $products = Product::where('aktif', true)-
>get();
        return view('products.index',
compact('products'));
    }
}
```

PERAN ORKESTRATOR

Controller bertindak seperti konduktor: tidak memainkan instrumen sendiri, tetapi memberi aba-aba kerja.

DEKLARATIF & RINGKAS

Perhatikan bagaimana method index() hanya bertugas mengambil koleksi model dan menyerahkannya ke view terkait.

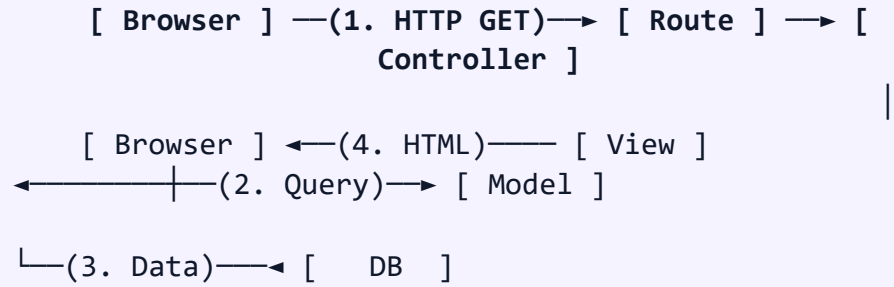
Alur Sirkulasi Data MVC (Step-by-Step)

Pemrograman dan Pengujian Aplikasi Web (12S3101) • Chandro Pardede, S.Kom., M.Sc.

Tahapan Transaksi Data

- 1. Pengguna memicu HTTP Request via browser.
- 2. Router mengarahkan request ke Controller spesifik.
- 3. Controller meminta data ke Model.
- 4. Model mengeksekusi query dan mengembalikan objek.
- 5. Controller mengirimkan objek data ke View.
- 6. View dikompilasi menjadi HTML bagi pengguna.

Diagram Alur Mental Sirkulasi Data



INTEGRITAS SIKLUS SATU ARAH

Setiap tahapan memiliki fungsi deterministik yang mempermudah pelacakan galat (*debugging*) saat terjadi anomali sistem.

PANDUAN PEMERIKSAAN KODE

Jika menemukan kode di mana View memanggil Model secara langsung, itu adalah tanda pelanggaran arsitektur MVC murni.

Miskonsepsi MVC: Fat Controller vs Fat Model

Bahaya Fat Controller Anti-Pattern

- Menumpuk ratusan baris query di Controller.
- Menuliskan logika pengiriman email di Controller.
- Menyatukan resize gambar di action method Controller.
- Menyebabkan kode sulit diuji secara modular.
- Memicu duplikasi logika saat membuat API seluler.

Kaidah Industri: Skinny Controller

- Pindahkan logika manipulasi query ke Model.
- Pindahkan pemrosesan bisnis kompleks ke *Service Classes*.
- Pindahkan logika validasi ke *Form Request Classes*.
- Controller tetap ramping, deklaratif, dan mudah dibaca.
- Meningkatkan reusabilitas kode antar platform.

GEJALA ANTI-PATTERN

Jika berkas controller Anda melampaui 300 baris kode, itu pertanda kuat terjadi pelanggaran Single Responsibility Principle.

ATURAN PRAKTIS DESAIN

Controller hanya bertugas mengkoordinasi: memvalidasi input, memanggil service atau model, dan mengembalikan response.

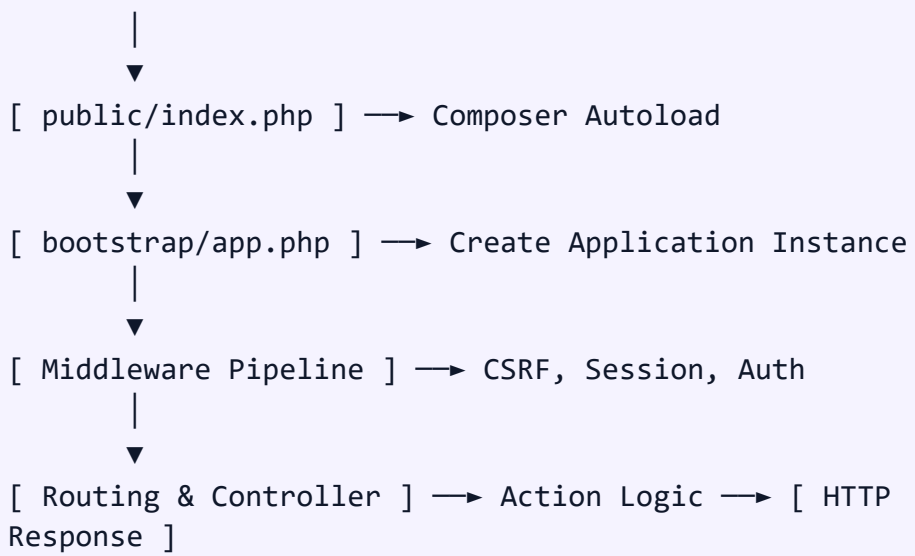
Request Lifecycle Overview di Laravel

Pemrograman dan Pengujian Aplikasi Web (12S3101) • Chandro Pardede, S.Kom., M.Sc.

Perjalanan Paket HTTP Masuk

- Titik awal: Server web Nginx atau Apache.
- Pintu gerbang tunggal: `public/index.php`.
- Inisialisasi autoloader Composer pihak ketiga.
- Pemuatan instansiasi Service Container framework.
- Penyaringan request melalui tumpukan Middleware.

Alur Jalur Eksekusi Internal



PENTINGNYA MEMAHAMI LIFECYCLE

Memahami alur lifecycle adalah pembeda utama antara sekadar programmer pengetik kode dan insinyur sistem perangkat lunak sejati.

DARI JARINGAN HINGGA RESPON

Seluruh proses ini dieksekusi dalam hitungan beberapa milidetik oleh engine internal PHP dan Laravel yang teroptimasi.

Entry Point: Peran public/index.php

Fungsi Strategis Berkas Entry

- Satu-satunya dokumen yang diekspos ke publik internet.
- Mengisolasi file konfigurasi rahasia dari akses luar.
- Mencatat waktu awal eksekusi sistem (*benchmark*).
- Mengikutsertakan autoloader Composer (`autoload.php`).
- Menangkap request global PHP menjadi objek request modern.

Anatomi Baris Kode public/index.php

```php
<?php
use Illuminate\Http\Request;

define('LARAVEL_START', microtime(true));

// 1. Memuat autoloader pustaka pihak ketiga
require __DIR__.'/../vendor/autoload.php';

// 2. Menjalankan bootstrap aplikasi & tangani request
(require_once __DIR__.'/../bootstrap/app.php')
    ->handleRequest(Request::capture());
```

KEAMANAN WEB SERVER

Mengarahkan root server ke folder `public/` mencegah pengunjung mengunduh file `.env` atau source code secara langsung.

FUNGSI Request::capture()

Mengonversi variabel global PHP ($_GET, $_POST, $_SERVER) menjadi objek Request berorientasi objek yang elegan.

Bootstrap Process: bootstrap/app.php

Inisialisasi Aplikasi Terpusat

- Membuat instance utama Service Container Laravel.
- Mendaftarkan berkas konfigurasi rute web dan API.
- Menetapkan tumpukan middleware global dan grup.
- Mengonfigurasi penanganan kesalahan (*exception handling*).
- Di Laravel 11, struktur kernel disederhanakan modular.

Struktur Deklaratif bootstrap/app.php

```
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath:
dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
    )
    ->withMiddleware(function (Middleware $middleware)
{
        // Konfigurasi middleware kustom
    })->create();
```

MODERNISASI LARAVEL 11

File Kernel.php yang kompleks pada versi terdahulu kini digantikan oleh pemanggilan rantai metode deklaratif di `bootstrap/app.php`.

PUSAT INTEGRASI SISTEM

Objek Application yang dikembalikan berkas ini adalah otak yang mengoordinasikan seluruh dependensi dan siklus hidup sistem.

Routing Engine: routes/web.php vs api.php

Pemisahan Karakteristik Rute

- `routes/web.php`: Rute dengan session, cookie, dan CSRF.
- `routes/api.php`: Rute stateless untuk integrasi API.
- Mendukung kata kerja HTTP: GET, POST, PUT, DELETE.
- Penamaan rute (*Named Routes*) untuk referensi dinamis.
- Pengelompokan rute (*Route Groups*) berdasarkan prefix atau middleware.

Deklarasi Rute Berstandar

```
use App\Http\Controllers\ProductController;
use Illuminate\Support\Facades\Route;

// Rute dasar dengan penamaan
Route::get('/products', [ProductController::class, 'index'])
    ->name('products.index');

Route::post('/products', [ProductController::class, 'store'])
    ->name('products.store');
```

MANFAAT NAMED ROUTES

Jika suatu saat URL diubah, pemanggilan via `route('products.index')` di seluruh aplikasi tidak akan pernah rusak.

KEAMANAN CSRF OTOMATIS

Seluruh rute yang terdaftar di web.php otomatis melewati middleware VerifyCsrfToken untuk mencegah serangan eksploitasi form.

Peran Middleware: Gatekeeper & Request Filter

Mekanisme Lapisan Pelindung

- Menerapkan pola arsitektur pipa (*Pipeline Pattern*).
- Mencegat dan memeriksa request sebelum masuk controller.
- Menghentikan request jika otentikasi tidak valid.
- Dapat memanipulasi header response sebelum dikirimkan.
- Contoh bawaan: verifikasi session, proteksi CSRF, enkripsi.

Struktur Middleware Kustom

```
namespace App\Http\Middleware;
use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserIsAdmin {
    public function handle(Request $request, Closure $next): Response {
        if (! $request->user()?->is_admin) {
            abort(403, 'Akses terbatas untuk administrator!');
        }
        return $next($request);
    }
```

PENJAGA PINTU GERBANG

Middleware adalah satpam aplikasi kalian yang memverifikasi hak akses sebelum pengguna diizinkan menyentuh controller.

CLOSURE $next($request)

Pemanggilan `$next($request)` adalah kunci yang meneruskan request ke middleware berikutnya atau langsung ke controller target.

Route Resolution: Implicit Model Binding

Pencocokan Rute Cerdas

- Pencocokan parameter dinamis dari segmentasi URL.
- Fitur *Route Model Binding* mengotomasi pencarian data.
- Mengeliminasi penulisan manual `Product::findOrFail($id)`.
- Otomatis melempar respons `404 Not Found` jika tiada.
- Mendukung pencarian kolom alternatif seperti slug atau NIM.

Manual vs Route Model Binding

```
// 1. PENDAFTARAN RUTE:
Route::get('/products/{product}',
[ProductController::class, 'show']);

// 2. CONTROLLER (Type-hint model Product):
public function show(Product $product): View {
    // $product otomatis terisi data lengkap dari
basis data!
    return view('products.show', compact('product'));
}
```

PENYEDERHANAAN KODE

Route Model Binding menghemat penulisan validasi eksistensi data di setiap action method controller.

PENANGANAN 404 OTOMATIS

Jika ID 999 tidak ditemukan di tabel products, Laravel langsung mengembalikan 404 tanpa mengeksekusi body method controller.

Peta Folder Laravel 11 (Root Overview)

Pemrograman dan Pengujian Aplikasi Web (12S3101) • Chandro Pardede, S.Kom., M.Sc.

Pengorganisasian Tingkat Atas

- `app/`: Rumah utama logika bisnis dan entitas data.
- `bootstrap/`: File booting framework dan cache sistem.
- `config/`: Berkas konfigurasi terpusat seluruh fitur.
- `database/`: Skema migrasi, seeder, dan factory data.
- `public/`: Web server root dan aset kompilasi statis.
- `resources/`: Template antarmuka Blade dan aset mentah.

Pohon Direktori Proyek`portal-del/`

```
├── app/              # Models, Controllers, Providers
├── bootstrap/        # app.php & cache
├── config/           # Seluruh berkas konfigurasi
├── database/         # Migrations, Seeders, Factories
├── public/           # Entry point & aset build
├── resources/        # Views (Blade), Raw CSS & JS
├── routes/           # web.php, console.php
├── storage/          # Logs, cache, file uploads
└── .env              # Variabel kredensial lingkungan
```

STANDARISASI STRUKTUR

Setiap developer Laravel di dunia mengenali struktur ini sehingga proses onboarding anggota tim baru berjalan sangat cepat.

JANGAN EDIT FOLDER VENDOR

Folder `vendor/` dikelola mutlak oleh Composer; modifikasi manual di dalamnya akan terhapus saat pembaruan paket.

Folder app/: Rumah Inti Logika Aplikasi

Pemrograman dan Pengujian Aplikasi Web (12S3101) • Chandro Pardede, S.Kom., M.Sc.

Subdirektori Utama di app/

- Menggunakan standar PSR-4 dengan namespace `App\`.
- `app/Http/Controllers/`: Logika penerima request HTTP.
- `app/Http/Middleware/`: Lapisan filter keamanan request.
- `app/Models/`: Kelas entitas data Eloquent ORM.
- `app/Providers/`: Pusat pendaftaran layanan container.
- Dapat diperluas dengan folder `Services/` atau `DTO/`.

Hierarki Folder app/app/

```
├── Http/
│   ├── Controllers/
│   │   ├── Controller.php
│   │   └── ProductController.php
│   └── Middleware/
├── Models/
│   ├── User.php
│   └── Product.php
└── Providers/
    └── AppServiceProvider.php
```

AREA KERJA UTAMA

Hampir seluruh kode fitur aplikasi yang Anda bangun akan berdomisili di dalam subfolder `app/` ini.

PEMISAHAN TANGGUNG JAWAB

Logika manipulasi data diletakkan di Models, sedangkan orkestrasi input-output diletakkan di Controllers.

Folder routes/ & config/: Jalur & Konfigurasi

Pusat Pengaturan Sistem

- `routes/web.php`: Peta seluruh URL antarmuka web.
- `routes/console.php`: Perintah terminal dan schedule cron.
- `config/`: Kumpulan array setelan resmi framework.
- Menggunakan helper `env()` untuk nilai fleksibel.
- Dilarang menulis kredensial hardcoded di file config.

Contoh Pembacaan Konfigurasi

```
// config/database.php
'default' => env('DB_CONNECTION', 'mysql'),

'connections' => [
    'mysql' => [
        'driver'   => 'mysql',
        'host'     => env('DB_HOST', '127.0.0.1'),
        'database' => env('DB_DATABASE',
'portal_del'),
    ],
],
```

KAIDAH KONFIGURASI AMAN

File konfigurasi bertugas mendefinisikan struktur, sedangkan nilai rahasia selalu diambil dari file `.env`.

PORTABILITAS LINGKUNGAN

Dengan memisahkan config dan .env, aplikasi dapat berpindah server tanpa perlu menyunting satu baris pun kode PHP.

Folder resources/: Aset Mentah & Antarmuka

Pusat Bahan Baku Frontend

- Menampung seluruh kode antarmuka sebelum dibuild.
- `resources/views/`: Berkas template antarmuka Blade.
- `resources/css/`: Berkas stylesheet mentah (Tailwind/Sass).
- `resources/js/`: Skrip modul JavaScript modern.
- Diproses oleh bundler Vite menjadi berkas produksi.

Struktur Berkas Antarmuka`resources/`

```
├── css/
│   └── app.css      # Styling Tailwind CSS
├── js/
│   ├── app.js       # Script JavaScript Utama
│   └── bootstrap.js # Konfigurasi Axios
└── views/
    ├── components/  # Komponen modular reusable
    ├── layouts/     # Master template shell
    └── products/    # Halaman fitur katalog
```

BELUM TERKOMPILASI

File di resources/ belum optimal untuk browser; Vite akan memadatkan ukurannya ke `public/build/`.

INTEGRASI VITE BUNDLER

Vite mengamati folder resources/ ini untuk melakukan reload instan saat Anda mengubah kode tampilan.

Folder database/: Kendali Versi Skema Data

Otomasi Skema & Data Uji

- `migrations/`: Skrip perancangan tabel tercatat waktu.
- Menggantikan pertukaran file `database.sql` manual.
- `factories/`: Cetak biru pembangun data acak realistis.
- `seeders/`: Pengisi data acuan awal sistem.
- Menjamin konsistensi skema antar seluruh anggota tim.

Hierarki Berkas Basis Data

```
database/
├── factories/
│   └── ProductFactory.php
├── migrations/
│   ├── 2026_10_01_000000_create_users_table.php
│   └── 2026_10_03_000000_create_products_table.php
└── seeders/
    └── DatabaseSeeder.php
```

MIGRATION ADALAH GIT BASIS DATA

Setiap perubahan kolom tercatat dalam file terpisah sehingga riwayat skema dapat ditelusuri dan di-rollback.

PERINTAH SINKRONISASI CEPAT

Cukup jalankan `php artisan migrate` saat menarik (*git pull*) kode terbaru dari repositori tim.

File .env: Manajemen Lingkungan & Kredensial

Isolasi Data Rahasia Server

- Menyimpan kredensial unik setiap komputer pengembang.
- Wajib dimasukkan ke dalam `.gitignore` demi keamanan.
- Berkas `.env.example` sebagai panduan struktur variabel tim.
- `APP_ENV`: Menentukan status lokal atau produksi.
- `APP_DEBUG`: Wajib bernilai `false` di server produksi.

Contoh Isi Konfigurasi .env

```
APP_NAME="Katalog Del"
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_katalog
DB_USERNAME=root
DB_PASSWORD=secret
```

BAHAYA KEBOCORAN KREDENSIAL

Jika file `.env` terunggah ke repositori publik GitHub, kredensial basis data Anda dapat diretas pihak luar.

PERINGATAN DEBUG MODE

Membiarkan APP_DEBUG=true di server produksi akan menampilkan username & password database saat terjadi error.

CLI Magic: Peran Artisan Console

Asisten Pengembang Baris Perintah

- CLI resmi bawaan Laravel berbasis Symfony Console.
- Mengotomasikan pembuatan berkas boilerplate kelas.
- Menjalankan migrasi dan seeding data database.
- Menyediakan terminal REPL interaktif via `tinker`.
- Memeriksa seluruh daftar rute via `route:list`.

Perintah Artisan Terpopuler

```
# 1. Generator lengkap (Model + Migration + Controller)
php artisan make:model Product -mcr

# 2. Menjalankan migrasi skema database
php artisan migrate

# 3. Menampilkan seluruh daftar rute terdaftar
php artisan route:list

# 4. Terminal eksperimen data interaktif
php artisan tinker
```

PRODUKTIVITAS MAKSIMAL

Insinyur perangkat lunak profesional jarang membuat file model atau controller secara manual dari tombol New File editor.

FLAG SAKTI -mcr

Flag -mcr menghasilkan berkas Model, berkas Migration, dan berkas Resource Controller sekaligus dalam 1 detik.

Desain Kasus: Modul Katalog Produk

Pemrograman dan Pengujian Aplikasi Web (12S3101) • Chandro Pardede, S.Kom., M.Sc.

Spesifikasi Modul Praktikum

- Membangun fitur manajemen katalog barang toko Del.
- Entitas data: id, kode, nama, harga, dan waktu.
- Menyediakan fungsi lihat daftar, tambah, dan validasi.
- Mengikuti alur standar arsitektur MVC terpadu.
- Menerapkan keamanan form dari serangan peretasan CSRF.

Peta Alur Komponen Kasus

```
1. database/migrations/   ⟶ Skema tabel 'products'
2. app/Models/Product.php ⟶ Objek data Eloquent
3. routes/web.php         ⟶ Rute Resource URL
4. ProductController.php  ⟶ Action index() & store()
5. resources/views/       ⟶ Form input & tabel Blade
```

TARGET IMPLEMENTASI

Kita akan menelusuri 7 langkah terstruktur mulai dari pembuatan database hingga tampilan antarmuka interaktif.

METODE BERTARAP

Setiap tahap saling bergantung secara logis; biasakan membangun lapisan data terlebih dahulu sebelum lapisan tampilan.

Step 1: Migration Skema Tabel Produk

Perancangan Tabel Basis Data

- Membuat file migrasi baru via perintah Artisan.
- Method `up()`: Menentukan kolom dan batasan integritas.
- Kolom `id()`: Kunci primer auto-increment bigint.
- Kolom `string('kode', 10)->unique()`: Kode barang unik.
- Method `down()`: Menghapus tabel saat proses rollback.

Isi File Migration (create_products_table)

```
public function up(): void {
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('kode', 10)->unique();
        $table->string('nama', 100);
        $table->unsignedInteger('harga');
        $table->timestamps(); // created_at & updated_at
    });
}

public function down(): void {
    Schema::dropIfExists('products');
```

EKSEKUSI PERINTAH

Jalankan perintah `php artisan migrate` di terminal untuk mengeksekusi pembuatan tabel ke database MySQL.

TIMESTAMP OTOMATIS

Kolom created_at dan updated_at diisi otomatis oleh Eloquent setiap kali data baru dibuat atau disunting.

Step 2: Model Eloquent (Product.php)

Konfigurasi Entitas Data

- Berkas terletak di `app/Models/Product.php`.
- Meng-extend kelas dasar `Illuminate\Database\Eloquent\Model`.
- Mendefinisikan array `$fillable` untuk mass assignment.
- Melindungi atribut sensitif dari injeksi formulir.
- Menyediakan fungsi bawaan query: `all()`, `find()`, `create()`.

Kode Lengkap Model Product

```php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    // Mencegah MassAssignmentException pada kolom:
    protected $fillable = [
        'kode',
        'nama',
        'harga'
    ];
}
```

KEAMANAN MASS ASSIGNMENT

Hanya kolom yang terdaftar di `$fillable` yang diizinkan disimpan melalui pemanggilan metode `Product::create()`.

OTOMATISASI KONEKSI TABEL

Karena nama kelas adalah Product, Eloquent secara cerdas menghubungkannya ke tabel products tanpa deklarasi manual.

Step 3: Mendefinisikan Rute Resource

Pendaftaran Endpoint RESTful

- Cukup deklarasikan 1 baris rute resource di `routes/web.php`.
- Otomatis memetakan 7 method standar CRUD.
- Mengaitkan rute langsung dengan `ProductController`.
- Standar URI seragam (`/products`, `/products/create`).
- Menghasilkan helper penamaan rute otomatis (`route('products.index')`).

Sintaks Rute Resource

```
use App\Http\Controllers\ProductController;
use Illuminate\Support\Facades\Route;

// Menghasilkan 7 aksi rute CRUD sekaligus!
Route::resource('products', ProductController::class);
```

DISIPLIN PENAMAAN ENDPOINT

Penggunaan rute resource menjamin arsitektur URL web Anda patuh pada konvensi RESTful kelas enterprise.

HASIL PEMETAAN RUTE

GET /products (index), POST /products (store), GET /products/{id} (show), PUT /products/{id} (update), DELETE (destroy).

Step 4: Controller Action & Query Model

Implementasi Method Aksi

- Method `index()`: Mengambil daftar produk terpaginasi.
- Method `store()`: Memvalidasi dan menyimpan rekaman baru.
- Memanfaatkan method `Product::create()` yang ekspresif.
- Mengembalikan redirect menuju halaman index produk.
- Mengirimkan flash message notifikasi keberhasilan transaksi.

Logika ProductController

```
public function index(): View
{
    $products = Product::latest()->paginate(10);
    return view('products.index',
compact('products'));
}

public function store(Request $request):
RedirectResponse {
    $valid = $request->validate([
        'kode'  => 'required|unique:products|max:10',
        'nama'  => 'required|min:3',
        'harga' => 'required|numeric|min:1000',
    ]);
    Product::create($valid);
```

PAGINASI BAWAAN

Pemanggilan `paginate(10)` otomatis menangani pembagian halaman dan kalkulasi offset query SQL secara efisien.

VALIDASI SEBELUM SIMPAN

Hanya data yang telah lulus validasi ($valid) yang dikirimkan ke database, menjamin integritas persistensi sistem.

Step 5: Menghubungkan Controller ke Blade View

Penyaluran Data Antar Layer

- Memanggil fungsi helper `view('products.index')`.
- Notasi titik memetakan direktori `views/products/index.blade.php`.
- Fungsi `compact('products')` menyuntikkan variabel data.
- Variabel `$products` siap dirender di template Blade.
- Menjaga keterpisahan total antara logika dan presentasi.

Sintaks Penyaluran Data

```php
// Opsi 1: Menggunakan fungsi compact (Praktis)
return view('products.index', compact('products'));

// Opsi 2: Menggunakan array asosiatif eksplisit
return view('products.index', [
    'products' => $products,
    'judul'    => 'Daftar Katalog Produk IT Del'
]);
```

DECOUPLING YANG KOKOH

View tidak peduli dari mana data berasal; View hanya fokus merender variabel `$products` yang disodorkan Controller.

VARIABEL TERSEDIA DI BLADE

Kunci array 'products' otomatis dapat diakses di template Blade menggunakan sintaks tanda dolar `$products`.

Step 6: View Rendering & Blade Syntax

Merender Antarmuka Produk

- Directive `@csrf`: Wajib disematkan pada setiap form POST.
- Directive `@foreach`: Melakukan iterasi daftar koleksi produk.
- Pencetakan variabel aman via tanda kurung ganda `{{ }}`.
- Menampilkan tautan paginasi via `{{ $products->links() }}`.
- Pesan error validasi ditampilkan menggunakan `@error`.

Formulir & Tabel Produk di Blade

```
<form action="{{ route('products.store') }}" method="POST">
    @csrf
    <input type="text" name="kode" value="{{ old('kode') }}">
    <input type="text" name="nama" value="{{ old('nama') }}">
    <input type="number" name="harga">
    <button type="submit">Simpan Produk</button>
</form>

@foreach ($products as $item)
    <p>{{ $item->kode }} - {{ $item->nama }} (Rp{{ $item->harga }})</p>
@endforeach
```

KEAMANAN CSRF FORM

Tanpa `@csrf`, peramban akan menerima respon penolakan HTTP 419 Page Expired saat mengirimkan formulir.

HELPER old()

Fungsi old('nama') menjaga agar isian teks formulir tidak hilang saat terjadi kesalahan validasi input.

Validasi Data: Form Request Validation

Isolasi Logika Validasi Kompleks

- Membuat kelas validasi dedikasi via Artisan CLI.
- Perintah: `php artisan make:request StoreProductRequest`.
- Memisahkan aturan validasi dari file controller utama.
- Otorisasi pengguna didefinisikan di method `authorize()`.
- Aturan validasi dideklarasikan di method `rules()`.

Contoh StoreProductRequest

```
namespace App\Http\Requests;
use Illuminate\Foundation\Http\FormRequest;

class StoreProductRequest extends FormRequest {
    public function authorize(): bool {
        return true; // Izinkan seluruh pengguna valid
    }
    public function rules(): array {
        return [
            'kode'  => 'required|unique:products|max:10',
            'nama'  => 'required|min:3',
            'harga' => 'required|numeric|min:1000',
        ];
```

CLEAN CODE ARCHITECTURE

Form Request Classes menjaga controller tetap ramping (*skinny*) dan mempermudah pengujian unit aturan validasi.

TYPE-HINT DI CONTROLLER

Ganti `Request $request` dengan `StoreProductRequest $request` di controller; validasi akan berjalan otomatis sebelum method dieksekusi!

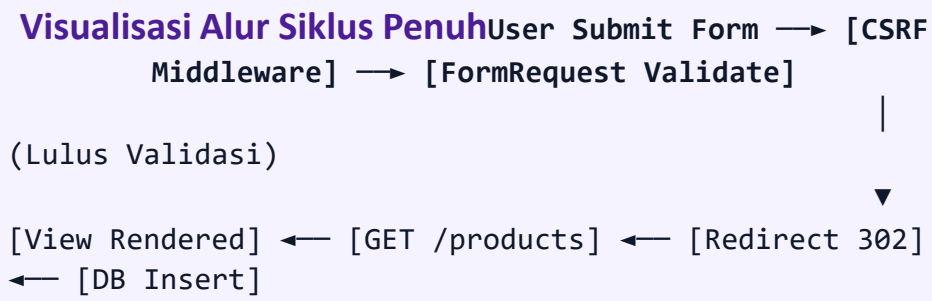
Diagram Alur Utuh Request CRUD (PRG Pattern)

Pemrograman dan Pengujian Aplikasi Web (12S3101) • Chandro Pardede, S.Kom., M.Sc.

Siklus Post-Redirect-Get (PRG)

- 1. Pengguna submit form: HTTP POST `/products`.
- 2. CSRF Middleware memvalidasi keaslian token sesi.
- 3. Form Request memverifikasi keabsahan data input.
- 4. Controller memanggil `Product::create()` ke DB.
- 5. Controller mengembalikan HTTP 302 Redirect.
- 6. Browser meminta HTTP GET `/products` (Pencegahan duplikasi).

Visualisasi Alur Siklus Penuh



MENGAPA HARUS REDIRECT?

Pola PRG mencegah pengguna melakukan submit form ganda saat menekan tombol refresh browser setelah simpan data.

SIKLUS TERINTEGRASI SEMPURNA

Seluruh komponen MVC berkolaborasi harmonis dalam alur ini: Route mengarahkan, Controller mengorkestrasi, Model memvalidasi & menyimpan, View merender tampilan akhir.

Local Development: Server Artisan

Lingkungan Kerja Lokal Praktis

- Menjalankan server lokal bawaan PHP CLI.
- Tidak mewajibkan instalasi Apache/Nginx saat tahap coding.
- Perintah eksekusi sederhana: `php artisan serve`.
- Server berjalan default di `http://127.0.0.1:8000`.
- Menampilkan log riwayat request secara langsung di terminal.

Output Terminal Artisan Serve

```
$ php artisan serve

   INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server

  2026-10-03 13:00:00
..............................  ~ 12ms
  2026-10-03 13:00:02 /products
.....................  ~ 24ms
```

PRAKTIS UNTUK LABORATORIUM

Mahasiswa dapat langsung mengembangkan aplikasi di laptop tanpa perlu konfigurasi VirtualHost server web yang rumit.

DUAL TERMINAL WORKFLOW

Buka Terminal 1 untuk menjalankan `php artisan serve`, dan buka Terminal 2 untuk menjalankan build frontend Vite.

Aset Frontend: Kompilasi Modern via Vite

Bundler Frontend Generasi Baru

- Vite menggantikan Webpack sebagai bundler default Laravel.
- Menyediakan fitur *Hot Module Replacement (HMR)* instan.
- Perubahan CSS/JS langsung ter-update tanpa reload halaman.
- Perintah pengembangan lokal: `npm run dev`.
- Berkas Blade memanggil aset via directive `@vite`.

Integrasi Vite di Layout Blade

```
<!-- Di Master Shell Layout (resources/views/layouts/app.blade.php) -->
<head>
    <title>Portal Produk IT Del</title>

    <!-- Directive Resmi Vite: -->
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
```

KECEPATAN NATIVE ES MODULES

Vite memulai dev-server dalam hitungan milidetik karena tidak perlu membundel seluruh file aset di awal tahap kerja.

OTOMATISASI ENVIRONMENT

Saat mode dev, directive @vite memuat skrip HMR lokal; saat mode produksi, ia otomatis memuat file minified dari public/build/.

Database Seeding & Testing Mock Data

Pemrograman dan Pengujian Aplikasi Web (12S3101) • Chandro Pardede, S.Kom., M.Sc.

Penyediaan Data Uji Cepat

- Mengisi puluhan data uji coba tabel secara otomatis.
- Memanfaatkan pustaka *Faker* untuk generate data realistis.
- Mempermudah pengujian fitur paginasi dan filter tabel.
- Perintah eksekusi seeder: `php artisan db:seed`.
- Reset total database: `php artisan migrate:fresh --seed`.

Implementasi Seeder Menggunakan Factory

```
// database/seeders/DatabaseSeeder.php
namespace Database\Seeders;
use App\Models\Product;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder {
    public function run(): void {
        // Otomatis membuat 50 data produk acak realistis!
        Product::factory(50)->create();
    }
}
```

PENGHEMAT WAKTU TESTING

Tidak perlu lagi mengisi 50 data produk manual satu per satu lewat form web hanya untuk menguji antarmuka paginasi.

SATU PERINTAH RESIK TOTAL

Perintah `php artisan migrate:fresh --seed` menghapus seluruh tabel lama, membangun skema baru, dan mengisinya dengan 50 data uji.

Production Preparation: Optimasi Cache

Menyiapkan Aplikasi Siap Produksi

- Mematikan pembacaan berkas konfigurasi berulang kali.
- `php artisan config:cache`: Menggabungkan seluruh setelan.
- `php artisan route:cache`: Mempercepat pencocokan rute sistem.
- `php artisan view:cache`: Mengompilasi seluruh template Blade.
- Satu perintah sapu jagat Laravel 11: `php artisan optimize`.

Perintah Optimasi Produksi

```
# 1. Melakukan kompilasi & caching seluruh komponen sistem
php artisan optimize

# Output:
# Configuration cached successfully.
# Routes cached successfully.
# Files cached successfully.

# 2. Membersihkan kembali cache saat ingin mengubah konfigurasi
php artisan optimize:clear
```

PERFORMA SERVIS SKALA BESAR

Caching konfigurasi dan rute dapat melipatgandakan kecepatan throughput request aplikasi hingga 300% di server cloud.

PENTING: CLEAR SETELAH EDIT

Jika Anda mengubah nilai di file .env setelah menjalankan optimize, jalankan optimize:clear agar perubahan tersebut terbaca.

Asset Bundling: npm run build & public/build/

Kompilasi Aset Frontend Produksi

- Mengompresi seluruh file CSS dan JS (*minification*).
- Menghapus kode CSS yang tidak terpakai (*tree-shaking*).
- Menyematkan kode hash anti-cache pada nama berkas.
- Menerbitkan output ke direktori `public/build/`.
- Server produksi tidak membutuhkan Node.js runtime lagi.

Output Terminal npm run build

```
$ npm run build

vite v5.4.0 building for production...
✓ built in 420ms

public/build/assets/app-B1z9k3.css   14.20 kB │ gzip:
3.10 kB
public/build/assets/app-C7x4m1.js    28.40 kB │ gzip:
9.80 kB
public/build/manifest.json            0.32 kB
```

MEKANISME CACHE BUSTING

Nama berkas seperti `app-B1z9k3.css` menjamin browser pengunjung tidak akan pernah memakai cache CSS lama setelah update situs.

HASIL ASET STATIS MURNI

File-file di public/build/ kini siap dilayani langsung oleh server web Nginx/Apache dengan performa loading super cepat.

Key Takeaways & Best Practices MVC

Pemrograman dan Pengujian Aplikasi Web (12S3101) • Chandro Pardede, S.Kom., M.Sc.

Prinsip Emas Pengembangan

- Pola MVC memisahkan data, tampilan, dan orkestrasi alur.
- Kaidah *Skinny Controller, Fat Model* menjaga keterawatan.
- Selalu terapkan validasi formulir di sisi server backend.
- Gunakan Route Model Binding untuk menyederhanakan controller.
- Manfaatkan Migration & Seeder untuk version control data.

Checklist Kualitas Kode Mahasiswa

- [✓] Model menggunakan `$fillable` (anti mass assignment).
- [✓] Seluruh form POST menyertakan directive `@csrf`.
- [✓] Rute menggunakan penamaan rute standar (`->name()`).
- [✓] Tidak ada query basis data mentah di berkas Blade View.
- [✓] Mode debug dinonaktifkan (`APP_DEBUG=false`) di produksi.

DISIPLIN REKAYASA SISTEM

Kerangka kerja modern menuntut kedisiplinan pengembang agar aplikasi tetap modular, aman, dan mudah diuji secara otomatis.

BEKAL KARIR SOFTWARE ENGINEER

Pemahaman alur HTTP dan MVC adalah fondasi universal yang dapat diterapkan pada framework apa pun di masa depan.

Q&A, Referensi Resmi, & Tugas Lab Minggu 5

Pemrograman dan Pengujian Aplikasi Web (12S3101) • Chandro Pardede, S.Kom., M.Sc.

Tugas Praktikum Mandiri Laboratorium Minggu 5:

[✓] Buat proyek baru Laravel 11 dengan nama `portal-akademik-del`.

[✓] Rancang Migration untuk tabel `students` (nim, nama, email, ipk).

[✓] Bangun kelas Model `Student` lengkap dengan konfigurasi `$fillable`.

[✓] Implementasikan `StudentController` menggunakan pola Resource Controller.

[✓] Buat antarmuka Blade untuk form pendaftaran dan tabel daftar mahasiswa.

DAFTAR REFERENSI AKADEMIK:

• Laravel Official Documentation (11.x). https://laravel.com/docs/11.x

• Stauffer, M. (2019). *Laravel: Up & Running (2nd ed.)*. O'Reilly Media.

• Martin, R. C. (2017). *Clean Architecture*. Prentice Hall.

• Fowler, M. (2002). *Patterns of Enterprise Application Architecture*.

Pertanyaan & Diskusi: chandropardede@del.ac.id • Institut Teknologi Del